TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

CẤU TRÚC DỮ LIỆU NÂNG CAO

TỪ CON SỐ 0 ĐẾN ỨNG DỤNG TRONG THI ĐẤU

Ngây thơ → Bế tắc → Lóe sáng → Tối ưu

NỘI DUNG

Mảng cộng dồn · Mảng hiệu · DSU · Sparse table
Fenwick (BIT) · Segment tree · Lazy propagation
Segment tree động · Persistent · Merge sort tree
Trie · Trie nhị phân (XOR) · Chia căn · Thuật toán Mo
LCA · Euler tour · HLD · Small to large

Lê Thành Nam

Lê Thành Nam

MỤC LỤC

GIỚI THIỆU 6
Tài liệu này dành cho ai? 6
Con số 2133 và con số 7 6
"Cấu trúc dữ liệu nâng cao" thực chất là gì? 6
Ôn lại: đọc độ phức tạp cho đúng 7
Khung chương trình dùng chung 7
Cách đọc tài liệu này 8
Phần 1. Khởi động: mảng cộng dồn và mảng hiệu 9
1.1. Bài toán 9
Ngây thơ 9
Bế tắc 9
Lóe sáng 9
Tối ưu 9
1.2. Mở rộng lên hai chiều 10
1.3. Chiều ngược lại: mảng hiệu 11
Ngây thơ và bế tắc 11
Lóe sáng 11
Tối ưu 11
1.4. Khi nào dùng gì - bảng quyết định 12
2.1. Bài toán mạng bạn bè 13
Ngây thơ 13
Bế tắc 13
Lóe sáng 13
Cài đặt thô - và vấn đề còn lại 14
2.2. Mẹo 1: ghép theo kích thước 14
2.3. Mẹo 2: nén đường đi 14
2.4. Độ phức tạp: con số kì lạ nhất trong tin học 15
2.5. Cài đặt hoàn chỉnh - bản dùng trong phòng thi 15
Chạy thử 15
2.6. Ứng dụng kinh điển: thuật toán Kruskal 16
2.7. Kĩ thuật nâng cao: chạy ngược để xoá cạnh 17
2.8. Những bài DSU thường gặp trong đề thi 18
2.9. Bẫy hay gặp 18
Phần 3. Sparse Table - Truy vấn min/max trong O(1) 19
3.1. Vì sao mảng cộng dồn không dùng được cho min 19
3.2. Bài toán RMQ 19
Ngây thơ 19

Lê Thành Nam

Ý tưởng ngây thơ thứ hai - và vì sao nó thất bại 19
3.3. Lóe sáng: chỉ chuẩn bị các đoạn dài luỹ thừa của 2 19
3.4. Mẹo hay nhất: cho phép chồng lấn 20
Minh chứng: vì sao TỔNG không làm được như vậy 20
3.5. Bảng log - chi tiết nhỏ quyết định tốc độ 20
3.6. Cài đặt hoàn chỉnh 21
3.7. Bảng so sánh với các cấu trúc khác 22
3.8. Những bài dùng sparse table 22
Phần 4. Fenwick Tree (BIT) - Tám dòng code thần thánh 23
4.1. Bài toán 23
Bế tắc - lần này thật sự khó 23
4.2. Lóe sáng: gộp một phần thôi 23
4.3. Phép thuật lowbit 23
Quy ước: mỗi ô phụ trách đoạn nào 24
4.4. Hai hàm, mỗi hàm ba dòng 24
query(i) - tổng từ 1 tới i 24
update(i, v) - cộng v vào a[i] 25
Truy vấn đoạn 25
4.5. Cài đặt hoàn chỉnh 25
4.6. Vì sao BIT nhanh hơn segment tree tới 7 lần 26
4.7. Ứng dụng 1: đếm nghịch thế 26
4.8. Ứng dụng 2: cập nhật đoạn - truy vấn điểm 27
4.9. Ứng dụng 3: cập nhật đoạn - truy vấn đoạn 27
4.10. Ứng dụng 4: tìm phần tử thứ k trong O(log n) 28
4.11. BIT hai chiều 28
4.12. Bảng bẫy của BIT 29
Bẫy 1 chi tiết - vì sao chỉ số 0 gây treo máy 29
Bẫy 7 chi tiết - vì sao BIT không làm được min 29
4.13. Bảng tổng kết BIT 29
Phần 5. Segment Tree - Cây phân đoạn 31
5.1. Vì sao cần thêm một cấu trúc nữa 31
5.2. Ý tưởng: chia để trị, nhưng lưu lại kết quả 31
5.3. Lưu cây bằng mảng - không cần con trỏ 31
Vì sao mảng phải cỡ 4n, không phải 2n 32
5.4. Hàm build 32
5.5. Hàm query - ba trường hợp 33
5.6. Hàm update - cập nhật một điểm 33
5.7. Chương trình hoàn chỉnh 33
5.8. Đổi phép gộp: từ tổng sang min, max, GCD 34
5.9. Bẫy của segment tree 35

Lê Thành Nam

5.10. So sánh cuối phần 36
Phần 6. Lazy Propagation - Nghệ thuật lười biếng 37
6.1. Lóe sáng 37
6.2. Hợp đồng bất di bất dịch 37
6.3. Ba hàm phụ trợ 37
apply - nhận một tờ giấy nợ 37
push - trả nợ cho hai con 38
6.4. update - cập nhật cả đoạn 38
6.5. query - thêm đúng một dòng 38
6.6. Chương trình hoàn chỉnh: cộng đoạn - tổng đoạn 38
6.7. Biến thể: cây min với lazy 40
6.8. Biến thể khó: GÁN cả đoạn 40
6.9. Bảng công thức apply cho các bài thường gặp 41
6.10. Bẫy của lazy propagation 41
Phần 7. Segment Tree nâng cao 43
7.1. Đi xuống trên cây (segment tree descent) 43
Ngây thơ và bế tắc 43
Lóe sáng 43
7.2. Segment tree động (dynamic / implicit) 44
7.3. Segment tree bền vững (persistent) 45
Bế tắc 45
Lóe sáng - hai bước 45
Cài đặt 45
7.4. Merge sort tree 46
7.5. Bảng chọn cấu trúc 47
Phần 8. Trie - Cây tiền tố 48
8.1. Bài toán từ điển 48
Ngây thơ và bế tắc 48
Lóe sáng 48
8.2. Cài đặt bằng mảng - không dùng con trỏ 48
Chèn một từ 49
Tìm một từ 49
Đếm từ có tiền tố p 49
8.3. Ước lượng bộ nhớ - chỗ dễ chết nhất 50
8.4. Trie nhị phân - vũ khí bí mật cho XOR 50
Lóe sáng 50
8.5. Ứng dụng đẹp nhất: XOR đoạn con lớn nhất 51
8.6. Bảng ứng dụng của trie 52
Phần 9. Chia căn và thuật toán Mo 53
9.1. Ý tưởng chia khối 53

Lê Thành Nam

Vì sao chọn k = √n? 53
9.2. Cài đặt chia khối 53
9.3. Thuật toán Mo - sắp xếp lại câu hỏi 54
Lóe sáng - hai bước 54
Cài đặt 55
9.4. Những bài Mo giải được 55
9.5. So sánh ba triết lí 56
Phần 10. Cấu trúc dữ liệu trên cây 57
10.1. Binary lifting - nhảy 2^k bước 57
Ngây thơ và bế tắc 57
Lóe sáng 57
10.2. LCA - tổ tiên chung gần nhất 57
10.3. Ý tưởng lớn: biến cây thành mảng 59
Lóe sáng 59
10.4. HLD - truy vấn trên ĐƯỜNG ĐI 60
Ý tưởng 60
Cài đặt 61
10.5. Small to large - gộp từ nhỏ vào lớn 62
Lóe sáng 62
10.6. Bảng chọn công cụ cho bài trên cây 63
Phần 11. Kĩ thuật thi đấu 64
11.1. Nén toạ độ - cây cầu bắt buộc 64
11.2. Cặp bài trùng kinh điển: nén toạ độ + BIT 64
11.3. Đếm cặp theo chiều ngược 65
11.4. Offline - vũ khí bị đánh giá thấp 65
11.5. Sắp xếp truy vấn theo giá trị 66
11.6. Bảng ra quyết định - dùng trong phòng thi 66
Phần 12. Tra cứu 68
12.1. Bảng độ phức tạp tổng hợp 68
12.2. Danh sách 20 bẫy - đọc trước khi nộp bài 68
12.3. Sổ tay cú pháp - dán lên tường 69
12.4. Mười lăm bài tập theo thứ tự tăng dần 70
12.5. Kế hoạch học tập 71
12.6. Ba câu để lại 71

Lê Thành Nam

GIỚI THIỆU

Tài liệu này dành cho ai?

Cho bạn - người đã biết viết một chương trình C++ chạy được, đã dùng được vector và sort, và vừa gặp bài đầu tiên khiến mình bó tay không phải vì "không nghĩ ra cách" mà vì "nghĩ ra rồi nhưng chạy quá chậm".

Đó là một cảm giác rất đặc trưng. Bạn đọc đề, hiểu ngay phải làm gì. Bạn viết code, chạy thử với ví dụ trong đề - đúng hết. Bạn nộp, và nhận về một dòng chữ lạnh lùng: Time Limit Exceeded. Thuật toán của bạn không sai. Nó chỉ chậm.

Cấu trúc dữ liệu nâng cao chính là câu trả lời cho tình huống đó. Không phải để nghĩ ra ý tưởng mới, mà để thực hiện đúng cái ý tưởng bạn đã có - nhanh hơn hàng trăm lần.

> **GHI NHỚ** **Tài liệu này không giả định bạn biết bất cứ thứ gì ngoài C++ cơ bản.** Mọi cấu trúc đều được xây từ một bài toán cụ thể, đi qua cách làm ngây thơ, chỉ ra chỗ bế tắc, rồi mới đến ý tưởng cứu nguy. Nếu bạn thấy một đoạn khó, đừng bỏ qua - hãy đọc lại phần "ngây thơ" ngay phía trên nó.

Con số 2133 và con số 7

Trước khi nói lí thuyết, hãy nhìn một con số thật.

Bài toán: cho mảng n = 200 000 số. Có q = 200 000 thao tác, mỗi thao tác là một trong hai loại: đổi giá trị một phần tử, hoặc hỏi tổng một đoạn. Đây là bài toán đơn giản nhất trên đời - không có mẹo, không có bẫy.

Mình đã viết ba chương trình giải bài này và bấm giờ thật trên máy:

```
Ngay tho (quet O(n)/truy van): 2133 ms   [chk=4446942]
Fenwick (BIT):                    7 ms   [chk=4446942]
Segment tree:                    49 ms   [chk=4446942]
```

Cả ba cho **cùng một kết quả** (`chk` giống hệt nhau - mình cố tình in ra để chứng minh không có gian lận). Chỉ khác tốc độ: cách ngây thơ chậm hơn Fenwick **hơn 300 lần**.

Với giới hạn thời gian 1 giây thường gặp trong đề thi, cách ngây thơ trượt, hai cách còn lại thừa sức. Toàn bộ cuốn tài liệu này là để bạn hiểu điều gì tạo ra khoảng cách 300 lần đó.

> **MẸO THI** **Đừng học thuộc code.** Người học thuộc sẽ quên sau hai tuần. Hãy hiểu *vì sao* mỗi cấu trúc nhanh - khi hiểu rồi, bạn tự viết lại được từ đầu, kể cả lúc trong phòng thi căng thẳng.

"Cấu trúc dữ liệu nâng cao" thực chất là gì?

Nghe thì to tát, nhưng ý tưởng cốt lõi chỉ có một câu:

> **GHI NHỚ** **Chuẩn bị trước một số thông tin phụ, để sau này trả lời câu hỏi mà không phải xem lại toàn bộ dữ liệu.**

Bạn đã dùng ý tưởng này rồi mà không biết. Khi bạn sắp xếp mảng để dùng lower_bound, chính là bạn đã "chuẩn bị trước" (sắp xếp) để "trả lời nhanh" (tìm kiếm nhị phân). Cấu trúc dữ liệu nâng cao chỉ là ý tưởng đó, được đẩy đi xa hơn.

Mọi cấu trúc trong tài liệu này đều nằm trên cùng một trục đánh đổi:

Cách làm	Chuẩn bị	Cập nhật	Truy vấn	Dùng khi
Mảng thường	O(1)	`O(1)`	O(n)	Rất ít truy vấn
Mảng cộng dồn	O(n)	O(n)	`O(1)`	Không bao giờ cập nhật
Fenwick / Segment tree	O(n)	`O(log n)`	`O(log n)`	Vừa cập nhật vừa hỏi
Sparse table	O(n log n)	- (không được)	`O(1)`	Không cập nhật, hỏi min/max

Nhìn bảng này bạn sẽ thấy ngay quy luật: không có cấu trúc nào thắng ở mọi cột. Mảng cộng dồn truy vấn O(1) nhưng cập nhật thảm hoạ. Fenwick không thắng ở cột nào cả, nhưng cũng không thua ở cột nào - và đó chính là lí do nó hữu dụng nhất.

> **MẸO THI** **Câu hỏi đầu tiên khi gặp một bài: "Dữ liệu có thay đổi không?"** Nếu KHÔNG → mảng cộng dồn hoặc sparse table, cực nhanh và cực dễ viết. Nếu CÓ → Fenwick hoặc segment tree. Trả lời đúng câu này là bạn đã đi được nửa đường.

Ôn lại: đọc độ phức tạp cho đúng

Bạn có thể đã học Big-O rồi, nhưng ở đây ta cần dùng nó theo cách rất thực tế: để đoán xem code có kịp giờ không, ngay trước khi gõ dòng đầu tiên.

Máy chấm thi thường chạy khoảng 10^8 phép tính đơn giản trong 1 giây. Từ đó suy ra bảng sau - hãy học thuộc nó, đây là bảng quan trọng nhất trong nghề:

n tối đa	Độ phức tạp cho phép	Thuật toán tiêu biểu
n ≤ 10	`O(n!)`	Duyệt mọi hoán vị
n ≤ 20	`O(2^n · n)`	Quy hoạch động bitmask
n ≤ 500	`O(n³)`	Floyd, nhân ma trận
n ≤ 5 000	`O(n²)`	QHĐ 2 chiều, duyệt cặp
n ≤ 10^5	`O(n log n)`	**Fenwick, segment tree, sort**
n ≤ 10^6	`O(n)`	Hai con trỏ, mảng cộng dồn
n ≤ 10^{18}	`O(log n)`	Luỹ thừa nhanh, tìm kiếm nhị phân

> **⚠ BẪY** **Dòng n ≤ 10^5 là dòng của tài liệu này.** Khi bạn thấy đề ghi n ≤ 10^5 hoặc $2 \cdot 10^5$ và có cả cập nhật lẫn truy vấn, gần như chắc chắn đề đang đòi một cấu trúc dữ liệu nâng cao. Đó là tín hiệu rõ ràng nhất.

Vì sao `O(log n)` lại thần kì đến vậy? Vì $\log_2$ tăng cực chậm. Với n = 10^5 thì $\log_2 n \approx 17$. Với n = 10^9 thì $\log_2 n \approx 30$. Nghĩa là dữ liệu tăng gấp 10 000 lần, công sức chỉ tăng chưa tới 2 lần. Đây là lí do mọi cấu trúc trong tài liệu này đều cố "chia đôi" bài toán.

Khung chương trình dùng chung

Mọi bài trong tài liệu này đều dùng khung sau. Hãy gõ nó thành thói quen:

```
#include <bits/stdc++.h>
```

```cpp
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);   // tắt đồng bộ với C -> cin/cout nhanh hơn nhiều
    cin.tie(nullptr);                   // cin không phải chờ cout xả

    // Nếu đề yêu cầu đọc/ghi file, mở dòng dưới ra:
    // freopen("BAI.INP", "r", stdin);
    // freopen("BAI.OUT", "w", stdout);

    // ... code ở đây ...
    return 0;
}
```

> ⚠ **BẪY** Hai dòng `sync_with_stdio` và `cin.tie` **không phải trang trí.** Với $2 \cdot 10^5$ dòng dữ liệu vào, thiếu chúng có thể chậm hơn 5–10 lần và đủ để trượt Time Limit dù thuật toán hoàn toàn đúng.

Khi đã bật `sync_with_stdio(false)`, tuyệt đối không trộn `printf/scanf` với `cin/cout` trong cùng chương trình - thứ tự in ra sẽ loạn.

Cách đọc tài liệu này

Mỗi phần lớn đều đi theo đúng một mạch bốn bước, lặp lại từ đầu đến cuối:

- **Ngây thơ.** Cách nghĩ tự nhiên nhất, ai cũng nghĩ ra. Nó ĐÚNG, chỉ chậm.
- **Bế tắc.** Tính ra con số cụ thể để thấy nó chậm cỡ nào, và chậm vì lí do gì.
- **Lóe sáng.** Một quan sát nhỏ lật ngược tình thế. Đây là phần quan trọng nhất, đọc chậm thôi.
- **Tối ưu.** Code hoàn chỉnh, chạy được, kèm kết quả chạy thật.

Nếu bạn chỉ đọc phần "Tối ưu" và chép code, bạn sẽ viết được đúng bài đó và tắc ở bài tiếp theo. Sức mạnh nằm ở bước "Lóe sáng".

> **GHI NHỚ** **Mọi đoạn code trong tài liệu này đều đã được biên dịch bằng g++ (C++17) và chạy thật.** Mọi kết quả in ra mà bạn thấy trong khung xám là kết quả máy thật in ra, không phải mình gõ tay. Nhiều đoạn còn được đối chiếu với cách làm ngây thơ trên dữ liệu ngẫu nhiên để chắc chắn không sai.